

# Advanced Natural Language Processing

## Lecture 03 Tekst and Document Representation

Department of Advanced Computer Sciences (DACS)  
Master of AI / DSDM  
Prof dr ir Jan C. Scholtes – Course year 2026-2027



Maastricht University

# Learning Objectives

- Compute TF-IDF weights and explain why raw term frequency and raw document frequency each need dampening (log weighting)
- Explain why cosine similarity, not Euclidean distance, is the right measure for comparing documents of different lengths
- Explain why bag-of-words / TF-IDF representations are extremely sparse and cannot capture word order
- Trace the evolution from static word embeddings (Word2Vec) to contextual embeddings (ELMo, BERT), and explain what problem each step solves
- Explain how WordPiece subword tokenization and multilingual/cross-lingual embeddings extend these representations beyond a single fixed vocabulary or language



# How do computer deal with text?

- What happens exactly when someone uses a computer program to deal with natural language? Computers are digital machines with limited capabilities.
- Computers cope best with numbers, in particular whole numbers, also known as integers, if it has to be really fast.
- People are analogue, and human language is also analogue, full of dualities, ambiguity, inconsistencies, interference, errors and exceptions.
- If we search for something then we often think in concepts, senses and meanings, all areas in which a computer cannot directly deal with.



# How do computer search in text?

- For computers to be able to make a computationally efficient search in a large amount of text, the task first needs to be converted into a numerical problem that a computer can deal with.
- This leads to very large containers containing many numbers in which numbers representing search terms that are compared with numbers representing documents and information.
- *This is the basic principle that our field concerns itself with: how can we translate information that we can work with into information that a computer can work with, and then translate the result back into a form that people can understand.*

# N-Grams (used in 1960's when memory was very limited) sub-word representation scheme.

- Represent a document as a vector.
- Each dimension holds occurrence frequency of n-gram in document.
- Example: 3-grams = 19.000 dimensional vector for case-insensitive English without numbers.

| FREQUENCIES FOR ICJA-JOURNAL TRIGRAMS |      |     |      |     |      |     |      |     |      |     |      |
|---------------------------------------|------|-----|------|-----|------|-----|------|-----|------|-----|------|
| tri                                   | freq | tri | freq | tri | freq | tri | freq | tri | freq | tri | freq |
| aba                                   | 297  | afv | 2    | akw | 5    | anv | 3    | ast | 434  | axe | 6    |
| abe                                   | 9    | afs | 14   | ak- | 39   | anw | 11   | asu | 155  | axi | 154  |
| abf                                   | 14   | aft | 188  | ala | 75   | any | 286  | asv | 4    | ax- | 28   |
| abi                                   | 58   | afw | 1    | alb | 39   | an- | 2083 | asy | 24   | aya | 6    |
| abl                                   | 790  | afz | 1    | alc | 13   | aoi | 1    | as- | 1621 | ayb | 2    |
| abo                                   | 215  | af- | 8    | ald | 8    | ao- | 3    | ata | 569  | aye | 141  |
| abr                                   | 6    | aga | 162  | ale | 467  | apa | 100  | atc | 68   | ayi | 61   |
| abs                                   | 100  | agd | 2    | alf | 34   | ape | 273  | ate | 1931 | aym | 5    |
| abu                                   | 21   | age | 455  | alg | 130  | aph | 75   | atg | 4    | ays | 81   |
| aby                                   | 2    | agi | 15   | ali | 285  | api | 40   | ath | 189  | ay- | 745  |
| ab-                                   | 5    | agl | 1    | alk | 5    | apl | 171  | ati | 2142 | aza | 14   |
| aca                                   | 1    | agn | 16   | alm | 24   | apo | 19   | atk | 57   | aze | 1    |
| ace                                   | 284  | ago | 12   | alo | 145  | apr | 184  | atl | 18   | azi | 17   |
| ach                                   | 645  | ah- | 129  | ip  | 2    | aps | 114  | atm | 8    | az- | 1    |
| aci                                   | 61   | ags | 4    | alr | 34   | apt | 9    | atn | 8    | a-- | 5039 |
| ack                                   | 722  | agu | 5    | als | 319  | apu | 9    | ato | 98   | bab | 24   |
| acl                                   | 14   | agy | 2    | alt | 94   | apy | 4    | atr | 75   | bac | 135  |
| acm                                   | 17   | ag- | 42   | alu | 314  | ap- | 68   | ats | 24   | bad | 32   |
| aco                                   | 93   | ahe | 9    | alv | 35   | aqu | 2    | atu | 151  | bag | 1    |
| acq                                   | 11   | ah- | 1    | alw | 30   | ara | 446  | aty | 12   | bak | 7    |
| acr                                   | 10   | aia | 1    | aly | 596  | arb | 46   | atz | 1    | bal | 30   |
| acs                                   | 1    | aic | 4    | al- | 2455 | arc | 651  | at- | 1691 | ban | 42   |
| act                                   | 609  | aid | 62   | ama | 8    | ard | 397  | aud | 8    | bar | 34   |
| acu                                   | 57   | aie | 3    | amb | 22   | are | 1144 | auf | 8    | bas | 596  |
| acy                                   | 14   | aig | 4    | ame | 1139 | arg | 101  | aug | 16   | bat | 23   |
| ac-                                   | 22   | ail | 157  | amh | 6    | ari | 317  | aul | 4    | bau | 4    |
| ada                                   | 43   | aim | 44   | ami | 118  | ark | 105  | aum | 2    | ba- | 10   |
| ade                                   | 208  | ain | 798  | amk | 1    | arl | 143  | aun | 6    | bco | 36   |
| adi                                   | 204  | aio | 2    | aml | 1    | arm | 17   | aus | 135  | bc- | 9    |
| adj                                   | 14   | air | 86   | amo | 101  | am  | 160  | aut | 104  | bdo | 13   |
| adl                                   | 13   | ais | 52   | amp | 222  | aro | 59   | auw | 1    | bd- | 29   |
| adm                                   | 48   | ait | 72   | amr | 2    | arp | 12   | au- | 10   | bea | 39   |
| ado                                   | 20   | aiv | 15   | ams | 158  | ars | 134  | ava | 93   | beb | 15   |
| adr                                   | 31   | ai- | 42   | amu | 31   | art | 562  | ave | 568  | bec | 119  |
| ads                                   | 34   | aja | 3    | amv | 4    | aru | 2    | avi | 135  | bed | 72   |
| adt                                   | 16   | aje | 4    | am- | 489  | arv | 3    | avo | 60   | bef | 64   |
| adu                                   | 2    | ajo | 40   | ana | 306  | arw | 2    | avr | 6    | beg | 37   |
| adv                                   | 237  | ajr | 11   | anc | 527  | arx | 6    | avs | 1    | beh | 43   |
| ady                                   | 38   | aka | 1    | and | 2787 | ary | 202  | avy | 1    | bei | 114  |
| ad-                                   | 342  | akb | 2    | ane | 58   | ar- | 404  | av- | 4    | bel | 5    |
| aef                                   | 28   | ake | 224  | ang | 226  | asa | 27   | awa | 30   | bel | 165  |
| aer                                   | 5    | akg | 2    | ani | 154  | asb | 3    | awb | 10   | ben | 347  |
| aes                                   | 1    | akh | 18   | ank | 201  | asc | 11   | awh | 1    | bep | 1    |
| ae-                                   | 7    | aki | 33   | anl | 3    | ase | 974  | awi | 4    | ber | 684  |
| afa                                   | 3    | akm | 1    | ann | 1    | asg | 23   | awl | 5    | bes | 125  |
| afe                                   | 49   | akn | 8    | ano | 179  | ash | 26   | awn | 394  | bet | 226  |
| afg                                   | 4    | aks | 1    | anq | 1    | asi | 304  | aws | 25   | beu | 1    |
| afh                                   | 1    | akt | 16   | ans | 343  | ask | 87   | aw- | 72   | bey | 50   |
| afl                                   | 3    | aku | 6    | ant | 538  | aso | 161  | axb | 3    | bez | 10   |
| afo                                   | 9    | akv | 3    | anu | 65   | asp | 37   | axd | 1    | be- | 1239 |

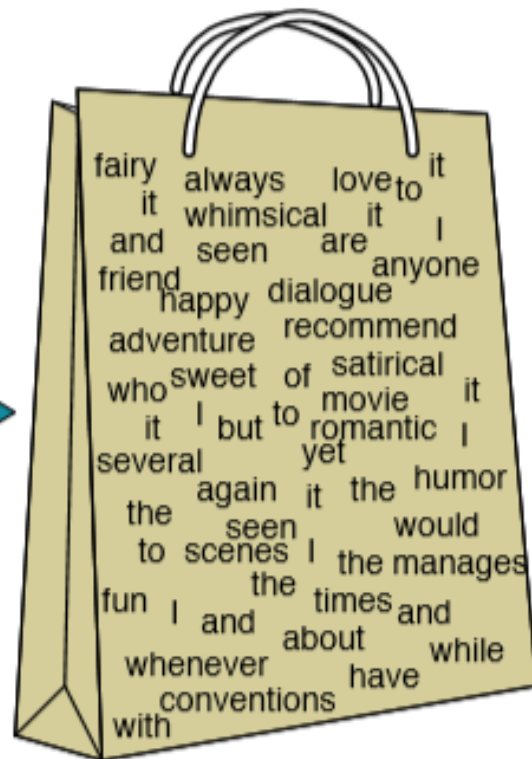
# Bag of Words (BoW)

|           | Antony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|-----------|----------------------|---------------|-------------|--------|---------|---------|
| Antony    | 1                    | 1             | 0           | 0      | 0       | 1       |
| Brutus    | 1                    | 1             | 0           | 1      | 0       | 0       |
| Caesar    | 1                    | 1             | 0           | 1      | 1       | 1       |
| Calpurnia | 0                    | 1             | 0           | 0      | 0       | 0       |
| Cleopatra | 1                    | 0             | 0           | 0      | 0       | 0       |
| mercy     | 1                    | 0             | 1           | 1      | 1       | 1       |
| worser    | 1                    | 0             | 1           | 1      | 1       | 0       |

***Brutus AND Caesar BUT NOT Calpurnia***

1 if play contains  
word, 0 otherwise

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



|           |     |
|-----------|-----|
| it        | 6   |
| I         | 5   |
| the       | 4   |
| to        | 3   |
| and       | 3   |
| seen      | 2   |
| yet       | 1   |
| would     | 1   |
| whimsical | 1   |
| times     | 1   |
| sweet     | 1   |
| satirical | 1   |
| adventure | 1   |
| genre     | 1   |
| fairy     | 1   |
| humor     | 1   |
| have      | 1   |
| great     | 1   |
| ...       | ... |

## Bag of Words Variation: Term-document count matrices

- Consider the number of occurrences of a term in a document:
  - Each document is a **count vector** in  $\mathbb{N}^v$ : a column below

|           | Antony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|-----------|----------------------|---------------|-------------|--------|---------|---------|
| Antony    | 157                  | 73            | 0           | 0      | 0       | 0       |
| Brutus    | 4                    | 157           | 0           | 1      | 0       | 0       |
| Caesar    | 232                  | 227           | 0           | 2      | 1       | 1       |
| Calpurnia | 0                    | 10            | 0           | 0      | 0       | 0       |
| Cleopatra | 57                   | 0             | 0           | 0      | 0       | 0       |
| mercy     | 2                    | 0             | 3           | 5      | 5       | 1       |
| worser    | 2                    | 0             | 1           | 1      | 1       | 0       |

# Term frequency tf

- The term frequency  $tf_{t,d}$  of term  $t$  in document  $d$  is defined as the number of times that  $t$  occurs in  $d$ .
- We want to use tf when computing query-document match scores. But how?
- Raw term frequency is not what we want:
  - A document with 10 occurrences of the term is more relevant than a document with 1 occurrence of the term.
  - But not 10 times more relevant.
- Relevance does not increase proportionally with term frequency.

**Teaching note:** Concrete illustration: a document with 1 occurrence and one with 10 shouldn't get a 10× relevance gap — that's why log-frequency weighting (next slide) compresses raw counts.

NB: frequency = count in IR

# Log-frequency weighting

- The log frequency weight of term  $t$  in  $d$  is

$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d}, & \text{if } \text{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

- $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 1000 \rightarrow 4$ , etc.

**Teaching note:** Notice the curve flattens fast: going from 10→1000 occurrences only adds 2 to the weight — this is exactly the dampening slide 8 asked for.

# Document frequency

- Rare terms are more informative than frequent terms
  - Recall stop words
- Consider a term in the query that is rare in the collection (e.g., *arachnocentric*)
- A document containing this term is very likely to be relevant to the query *arachnocentric*
- → We want a high weight for rare terms like *arachnocentric*.

# idf weight

- $df_t$  is the document frequency of  $t$ : the number of documents that contain  $t$ 
  - $df_t$  is an inverse measure of the informativeness of  $t$
  - $df_t \leq N$  ( $N = \text{total number of documents in collection}$ )
- We define the idf (inverse document frequency) of  $t$  by

$$idf_t = \log_{10} (N/df_t)$$

- We use  $\log (N/df_t)$  instead of  $N/df_t$  to “dampen” the effect of idf.

Will turn out the base of the log is immaterial.

# idf example, suppose $N = 1$ million

| term      | $df_t$    | $idf_t$ |
|-----------|-----------|---------|
| calpurnia | 1         |         |
| animal    | 100       |         |
| sunday    | 1,000     |         |
| fly       | 10,000    |         |
| under     | 100,000   |         |
| the       | 1,000,000 |         |

$$idf_t = \log_{10} (N/df_t)$$

There is one idf value for each term  $t$  in a collection.

# idf example, suppose $N = 1$ million

| term      | $df_t$    | $idf_t$ |
|-----------|-----------|---------|
| calpurnia | 1         | 6       |
| animal    | 100       | 4       |
| sunday    | 1,000     | 3       |
| fly       | 10,000    | 2       |
| under     | 100,000   | 1       |
| the       | 1,000,000 | 0       |

$$idf_t = \log_{10} (N/df_t)$$

There is one idf value for each term  $t$  in a collection.

**Teaching note:** Notice idf falls monotonically as df rises — “the” (in every document) gets idf = 0, contributing nothing to a query match.

# Collection vs. Document frequency

- The collection frequency of  $t$  is the number of occurrences of  $t$  in the collection, counting multiple occurrences.
- Example:

| Word             | Collection frequency | Document frequency |
|------------------|----------------------|--------------------|
| <i>insurance</i> | 10440                | 3997               |
| <i>try</i>       | 10422                | 8760               |

- Which word is a better search term (and should get a higher weight)?

# tf-idf weighting

- The tf-idf weight of a term is the product of its tf weight and its idf weight.

$$w_{t,d} = \log(1 + \text{tf}_{t,d}) \times \log_{10}(N / \text{df}_t)$$

- **Best known weighting scheme in information retrieval**
  - Note: the “-” in tf-idf is a hyphen, not a minus sign!
  - **Alternative names: tf.idf, tf x idf**
- Increases with the number of occurrences within a document
- **Increases with the rarity of the term in the collection**

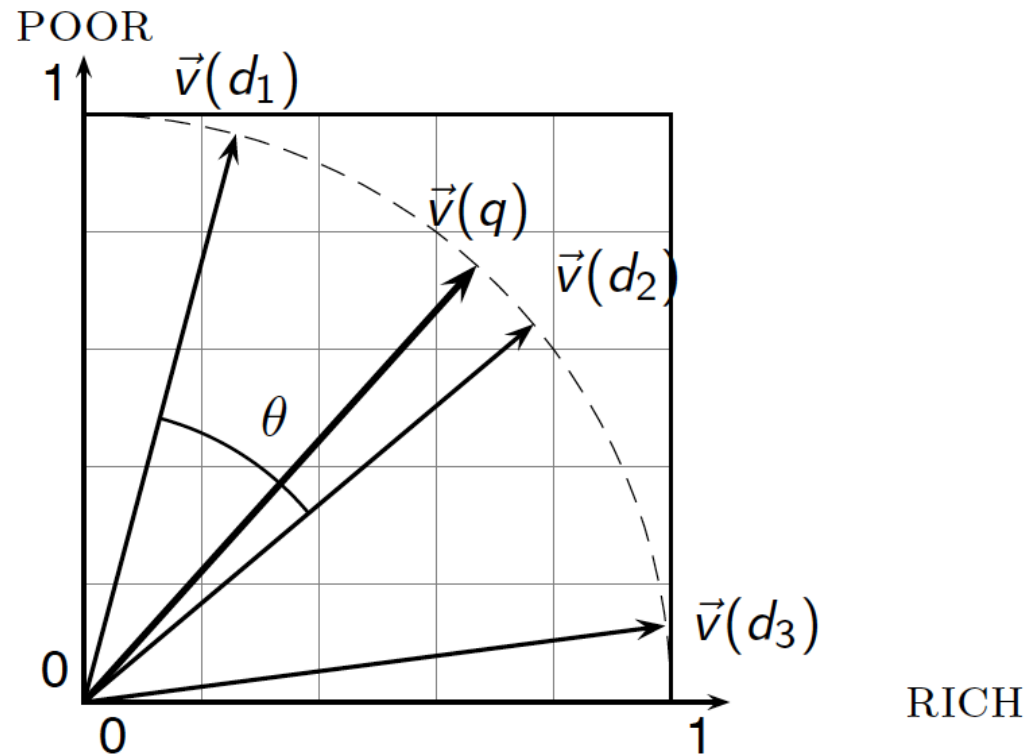
# Binary → count → weight matrix

**Teaching note:**  $\text{tf-idf}(t,d) = \text{tf}(t,d) \times \log(N / \text{df}(t))$  — rows are terms, columns are documents; high weight = frequent here, rare elsewhere.

|                  | Antony and Cleopatra | Julius Caesar | The Tempest | Hamlet | Othello | Macbeth |
|------------------|----------------------|---------------|-------------|--------|---------|---------|
| <b>Antony</b>    | 5.25                 | 3.18          | 0           | 0      | 0       | 0.35    |
| <b>Brutus</b>    | 1.21                 | 6.1           | 0           | 1      | 0       | 0       |
| <b>Caesar</b>    | 8.59                 | 2.54          | 0           | 1.51   | 0.25    | 0       |
| <b>Calpurnia</b> | 0                    | 1.54          | 0           | 0      | 0       | 0       |
| <b>Cleopatra</b> | 2.85                 | 0             | 0           | 0      | 0       | 0       |
| <b>mercy</b>     | 1.51                 | 0             | 1.9         | 0.12   | 5.25    | 0.88    |
| <b>worser</b>    | 1.37                 | 0             | 0.11        | 4.15   | 0.25    | 1.95    |

Each document is now represented by a real-valued vector of tf-idf weights  $\in \mathbb{R}^{|V|}$

# Documents as vectors – Document Collections as Matrices: similarity by measuring angles / cosines.



**Teaching note:**  $\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \|\mathbf{B}\|)$  — measures orientation, not magnitude, so document length doesn't skew similarity.

# Use angle instead of distance

- Thought experiment: take a document  $d$  and append it to itself. Call this document  $d'$ .
- “Semantically”  $d$  and  $d'$  have the same content
- The Euclidean distance between the two documents can be quite large
- The angle between the two documents is 0,  $\cos = 1 \Rightarrow$  corresponding to maximal similarity.  $\cos = 0$ , no similarity, orthogonal, 90 degrees.
- Key idea: Rank documents according to angle with query.

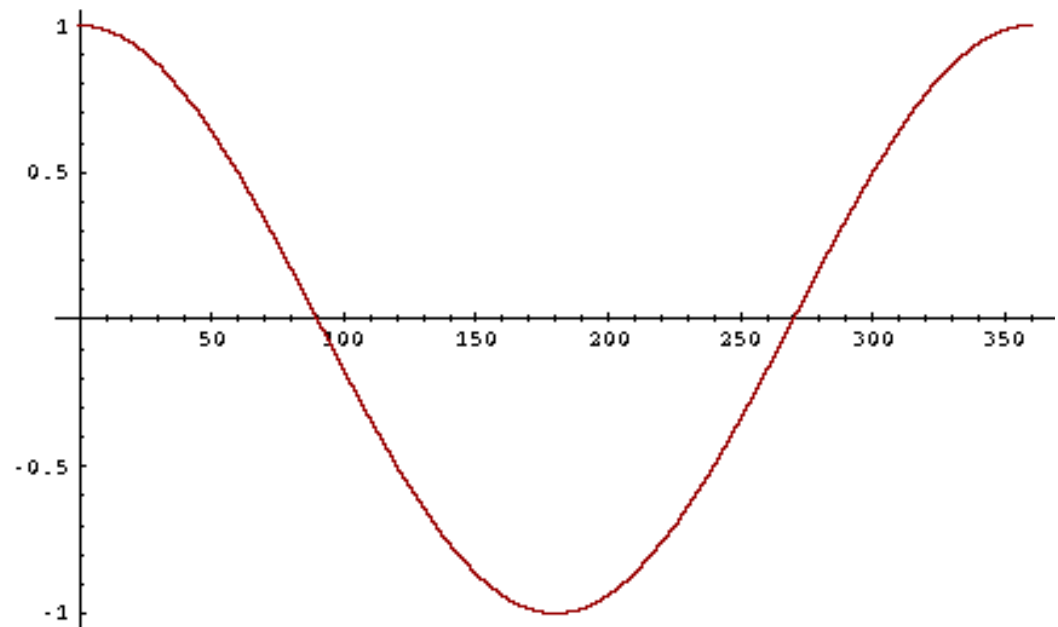
# From angles to cosines

- The following two texts are equivalent.
  - Rank text in decreasing order of the angle between query and document
  - Rank text in increasing order of  $\cos(\text{query}, \text{document})$
- Cosine is a monotonically decreasing function for the interval  $[0^\circ, 180^\circ]$

**Teaching note:** Because cosine decreases as angle increases, “rank by smallest angle” and “rank by largest cosine” give the identical ordering — that's why search engines just compute cosine directly.

# From angles to cosines

- Similar text  $\Rightarrow$  angle = 0, cosine = 1
- Orthogonal – no similarity in text  $\Rightarrow$  angle = 90 degrees, cosine = 0



# TF-IDF representations are very sparse

- TF-IDF values are the values in the dimensions of a vector. Each document or paragraph is then represented by such a vector.
- One dimension for each unique token.
- No document contains all tokens from a data set.
- Often  $<0.01\%$  is non-zero.
- But we have not taken word sequences into consideration...

**Teaching note:** This sparsity plus the missing word-order information are the two structural limits of TF-IDF — the next slide (word order) and word embeddings (later) each address one.

# How about word order?

- Lost with both TF-IDF and with Cosine distance.
- “Jane hit Jon” same vector / angle as “Jon hit Jane”

# Word Embeddings for transfer learning

Transfer Learning: understand basics of natural language (morphology, syntax, semantics, etc., and built on top of that to learn new more complex language tasks such as MT, NER, Q&A, etc.)



# Lexicon

- By being able to look up a word, we immediately know everything about that word:
  - Is it a verb, noun, adjective, or all of the above?
  - What are the possible inflexions
  - What are possible tenses
  - What is meaning?
  - Are there any proverbial forms?
  - ...

**Teaching note:** A lexicon only works if you already know the word's exact surface form — WordNet (next slide) adds relations between meanings, not just entries.



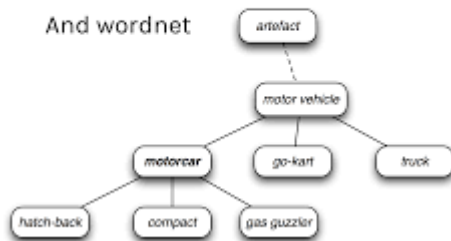
tends to yield to stronger personalities) — see WEAK 2  
 7 not harsh or stern especially in nature or effect (a *soft* breeze coming off the lake) — see GENTLE 1  
 8 providing physical comfort (a warm, *soft* bed to rest my weary bones) — see COMFORTABLE 1  
 9 tolerant and kind in the judgment of and expectations for others (thought they were too *soft* with their children) — see INDULGENT 1  
 10 marked by temperatures that are neither too high nor too low (It was a lovely *soft* spring evening.) — see CLEMENT 1  
**soften** *vb* 1 to diminish the physical strength of (Three weeks of being sick in bed had noticeably *softened* her.) — see WEAKEN 1  
 2 to lessen the shock of (The author's agent tried to *soften* the blow of the publisher's rejection.) — see CUSHION  
**softened** *adj* lacking bodily strength (The athlete, *softened* by the long period of convalescence, had to begin his training program almost from scratch.) — see WEAK 1  
**softhearted** *adj* having or marked by sympathy and consideration for others (a *softhearted* person who never hesitates to help anyone in trouble) — see HUMANE 1  
**softheartedness** *n* 1 the capacity for feeling for another's unhappiness or misfortune (the kind of *softheartedness* that makes him an easy target for anyone with a tale of woe) — see HEART 1  
 2 sympathetic concern for the well-being of others (Her *softheartedness* makes it hard for her to say "no" to charitable appeals.) — see BENIGNANCY  
**softness** *n* the quality or state of lacking strength of will or character (Softness is not a typical quality of military leaders.) — see WEAKNESS 2  
**soft-soap** *vb* 1 to get (someone) to do something by gentle urging, special attention, or flattery (She cunningly *soft-soaped* her parents into letting her go on the trip.) — see COAX  
 2 to praise too much (shrewd voters who know when a politician is trying to *soft-soap* them) — see FLATTER 1  
**soft soap** *n* excessive praise (a salesman who knows the value of *soft soap* in making a sale) — see FLATTERY  
**softy** or **softie** *n* a person lacking in physical strength (a *softy* who usually needs someone else's strong hands to open bottles and jars) — see WEAKLING 1  
**soggy** *adj* containing, covered with, or thoroughly penetrated by water (We'll spread the *soggy* papers out to dry.) — see WET 1  
<sup>1</sup> **soil** *n* foul matter that mars the purity or cleanliness of something (I got some sort of *soil* on my white pants.) — see FILTH 1  
<sup>2</sup> **soil** *n* 1 the loose surface material in which plants naturally grow (bought rich *soil* to plant flowers in) — see DIRT 1  
 2 the solid part of our planet's surface as distinguished from the sea and air (happy to have *soil* under my feet after that long sea voyage) — see EARTH 2  
**soil** *vb* to make dirty (Oil and grease *soiled* the mechanic's shirt.) — see DIRTY  
**soilage** *n* the state or quality of being dirty (The fee for cleaning the carpet will depend upon the extent of the *soilage*.) — see DIRTINESS  
**soiled** *adj* not clean (a *soiled* carpet in need of a good shampooing) — see DIRTY 1  
**sojourn** *n* a temporary residing as another's guest (Alex spent a relaxing *sojourn* in her friend's summer home.) — see VISIT 1  
**sojourn** *vb* to reside as a temporary guest (began their retirement by leisurely *sojourning* with friends and relatives scattered across the country) — see VISIT 2  
**solace** *n* 1 a feeling of ease from grief or trouble (The kind words brought a little *solace*.) — see COMFORT 1  
 2 the giving of hope and strength in times of grief, distress, or suffering (the selfless *solace* of the sick by the workers at the hospice) — see CONSOLATION 1  
**solace** *vb* 1 to ease the grief or distress of (Counselors did their best to *solace* the bereaved.) — see COMFORT 2  
 2 to cause (someone) to pass the time agreeably occupied (I *solaced* myself with a book while I waited for the bus.) — see AMUSE  
**solacing** *n* the giving of hope and strength in times of grief, distress, or suffering (I will be forever grateful for my friend's *solacing* of me when I was down.) — see CONSOLATION 1  
**solar plexus** *n* the part of the body between the chest and the pelvis (felt a sharp pain in the *solar plexus*) — see STOMACH 1  
**soldier** *n* a person engaged in military service (a platoon of *soldiers*)  
**synonyms** dogface, fighter, legionary, legionnaire, man-at-arms, regular, serviceman, warrior  
**related words** servicewoman; cavalier, cavalryman, cuirassier, dragoon; doughboy, footman, foot soldier, infantryman; commando, raider; marine, ranger; artilleryman, cannoner; gunner, musketeer, rifleman; GI, guardsman, militiaman, minuteman; conscript, draftee, enrollee, recruit; reservist  
**antonyms** civilian  
**soldierly** *adj* of, relating to, or suitable for war or a warrior (Noah Webster's brief contribution to the Revolutionary cause suggests that his *soldierly* skills were few.) — see MARTIAL 1  
**sole** *adj* 1 belonging only to the one person, unit, or group named (The landowner has *sole* rights to the property, so he can do whatever he wants to with it.)  
**synonyms** exclusive, single, unshared  
**related words** proprietary; personal, private  
**near antonyms** common, communal, conjoint, cooperative, joint, multiple, mutual, pooled, public, shared, united  
**antonyms** nonexclusive  
 2 being the one or ones of a class with no other members (The eldest son became the family's *sole* support.) — see ONLY 2  
**solecism** *n* a socially improper or unsuitable act or remark (the *solecism* of asking one's hosts how much something in their house cost them) — see IMPROPRIETY 2  
**solely** *adv* 1 for nothing other than (The promotion was based *solely* on merit.)  
**synonyms** alone, exclusively, just, only, purely, simply  
**related words** basically, by and large, chiefly, generally, largely, mainly, mostly, predominantly, primarily, principally, substantially  
**near antonyms** additionally, also, besides, likewise  
 2 without aid or support (You undertook that project *solely* on your own, and you will finish it likewise.) — see ALONE 1  
**solemn** *adj* 1 having or showing a formal and serious or reserved manner (The director of the funeral home has a fittingly *solemn* demeanor.) — see DIGNIFIED  
 2 not joking or playful in mood or manner (solemn as a judge) — see SERIOUS 1  
 3 causing or marked by an atmosphere lacking in cheer (The Capitol's rotunda was draped in *solemn* decorations of a state funeral.) — see GLOOMY 1  
**solemnity** *n* 1 a mental state free of jesting or trifling (The coronation ceremony requires absolute *solemnity*.) — see EARNESTNESS  
 2 an oft-repeated action or series of actions performed in accordance with tradition or a set of rules (Elaborate *solemnities* marked the 100th anniversary of the event.) — see RITE  
**solemnness** *n* a mental state free of jesting or trifling (I laid the wreath at the Tomb of the Unknown Soldier with all the *solemnness* that the occasion requires) — see EARNESTNESS

# Thesaurus or Ontology: Wordnet

- Meaning
- Associated words (opposite, similar meaning)



And wordnet



<https://wordnet.princeton.edu/>

**Teaching note:** Word embeddings (next slides) solve exactly this: they replace random hash-lookup addresses with vectors where distance reflects meaning.

## Hamburger

- Hamburger (an inhabitant of Hamburg)
  - direct hypernym:
    - German (a person of German nationality)
  - sister term
    - German (a person of German nationality)
      - East German (a native/inhabitant of the former GDR)
      - Bavarian (a native/inhabitant of Bavaria)
- derivationally related form
  - Hamburg (a port city in northern Germany on the Elbe River that was founded by Chalemagne in the...)

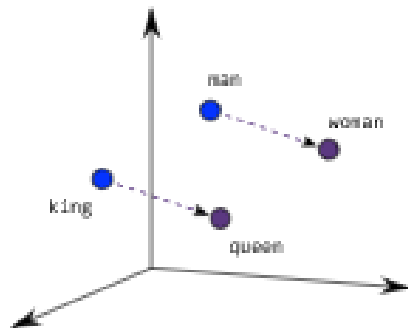
## The Problem:

- In traditional NLP systems, tokens are represented by hash-values, which can be used as lookup address.
- Hash values are random and have a flat distribution.
- No relation to a linguistic meaning.
- No problem in WordNet or rules-based approaches, but when used in deep-learning models: no relation between encoding and meaning.

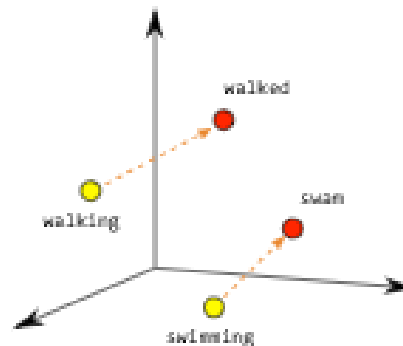
# Central Idea to Word Embeddings

- How can we assign an (high-dimensional) encoding to a token that represents morphological, syntactical, semantical and other linguistic properties of a token so that similar linguistic properties result in similar encodings?
- As a result, we should benefit from better linguistic representations and potentially transfer learning properties.

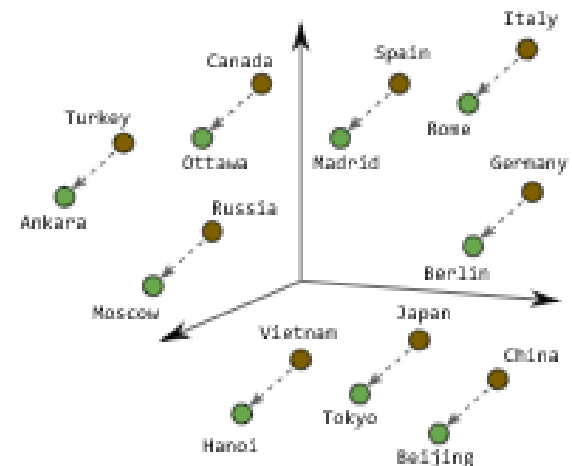
# Word Embeddings: similar linguistic context, similar numeric representations



Male-Female



Verb Tense



Country-Capital

**Teaching note:** Classic analogy:  $\text{vec}(\text{"king"}) - \text{vec}(\text{"man"}) + \text{vec}(\text{"woman"}) \approx \text{vec}(\text{"queen"})$ .  
Each panel is one semantic axis learned purely from co-occurrence: gender, verb tense, capital-country.

# Why Word Embeddings?

- Pre-trained language model.
- Understand context better.
- Transfer learning: understand already general aspects of language, subsequent only need to fine-tune for a specific NLP task.
- No need for millions or billions of annotated training data (when using deep learning) when learning a specific linguistic task.

# This goes back to Wittgenstein

In 1922 Ludwig Wittgenstein published the **Tractatus Logico-Philosophicus**.

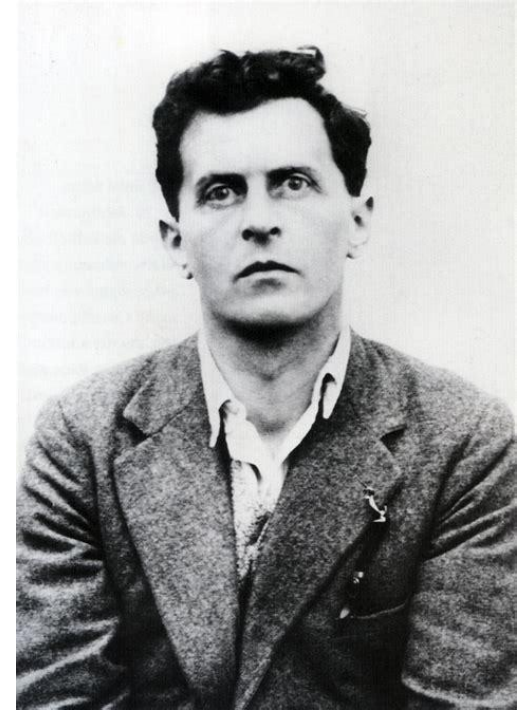
The project had a broad goal: to identify the relationship between language and reality and to define the limits of science.

He discusses the process of identifying a word's meaning from the context of its usage.

It can be argued that the Tractarian thoughts can be understood as acts of geometric projection in logical space, performed by the transcendental subject.

See also:

- Wittgenstein, Ludwig. 1953. *Philosophical Investigations*. John Wiley & Sons
- Tractatus Logico-Philosophicus, Logisch-philosophische Abhandlung By Ludwig Wittgenstein, First published by Kegan Paul (London), 1922.
- Projective Geometry in Logical Space: Rethinking Tractarian Thoughts Article in International Journal of Philosophical Studies · December 2017 DOI: 10.1080/09672559.2017.1404623 by Pablo Acuna.



**Teaching note:** Note: the source usually cited in NLP for “meaning from context” is J.R. Firth's distributional hypothesis (1957): “you shall know a word by the company it keeps.”

# Let's Start understanding Word2Vec (2013)

- We're going to train a simple (back propagation) neural network with a single hidden layer to perform a certain task, but then we're not actually going to use that neural network for the task we trained it on! Instead, the goal is actually just to learn the weights of the hidden layer—we'll see that these weights are actually the “word vectors” that we're trying to learn.
- Given a specific word in the middle of a sentence (the input word), look at the words nearby and pick one at random. The network is going to tell us the probability for every word in our vocabulary of being the “nearby word” that we chose.
- We will call this Skip-grams: a technique where by n-grams are still stored to model language, but they allow for tokens to be skipped to overcome the data sparsity problem. We analyze this by computing all possible skip-grams in a training corpus and measure how many adjacent (standard) n-grams these cover in test documents.

See also: Thomas Mikolov et al., Efficient Estimation of Word Representations in Vector Space (arXiv:1301.3781, January 2013), and Distributed Representations of Words and Phrases and their Compositionality (NIPS 2013)



# CBOW: Skip-grams

Continuous bag of words (CBOW) are modelled using Skip-grams:

| Source Text                                                                                     | Training Samples |       |       |                                                                  |
|-------------------------------------------------------------------------------------------------|------------------|-------|-------|------------------------------------------------------------------|
| <table><tr><td>The</td><td>quick</td><td>brown</td></tr></table> fox jumps over the lazy dog. ➡ | The              | quick | brown | (the, quick)<br>(the, brown)                                     |
| The                                                                                             | quick            | brown |       |                                                                  |
| The <table><tr><td>quick</td><td>brown</td><td>fox</td></tr></table> jumps over the lazy dog. ➡ | quick            | brown | fox   | (quick, the)<br>(quick, brown)<br>(quick, fox)                   |
| quick                                                                                           | brown            | fox   |       |                                                                  |
| The quick <table><tr><td>brown</td><td>fox</td><td>jumps</td></tr></table> over the lazy dog. ➡ | brown            | fox   | jumps | (brown, the)<br>(brown, quick)<br>(brown, fox)<br>(brown, jumps) |
| brown                                                                                           | fox              | jumps |       |                                                                  |
| The quick brown <table><tr><td>fox</td><td>jumps</td><td>over</td></tr></table> the lazy dog. ➡ | fox              | jumps | over  | (fox, quick)<br>(fox, brown)<br>(fox, jumps)<br>(fox, over)      |
| fox                                                                                             | jumps            | over  |       |                                                                  |

Skip-grams allow the language model to take additional context into account

## Why Softmax?

Input Vector



A '1' in the position  
corresponding to the  
word "ants"

A so-called  
Hot neuron

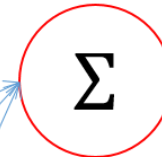
10,000  
positions

Hidden Layer  
Linear Neurons

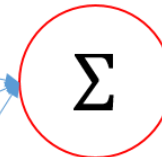


300 neurons

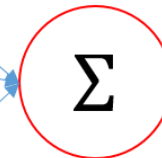
Output Layer  
Softmax Classifier



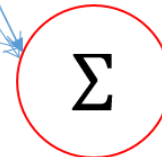
Probability that the word at a  
randomly chosen, nearby  
position is "abandon"



... "ability"



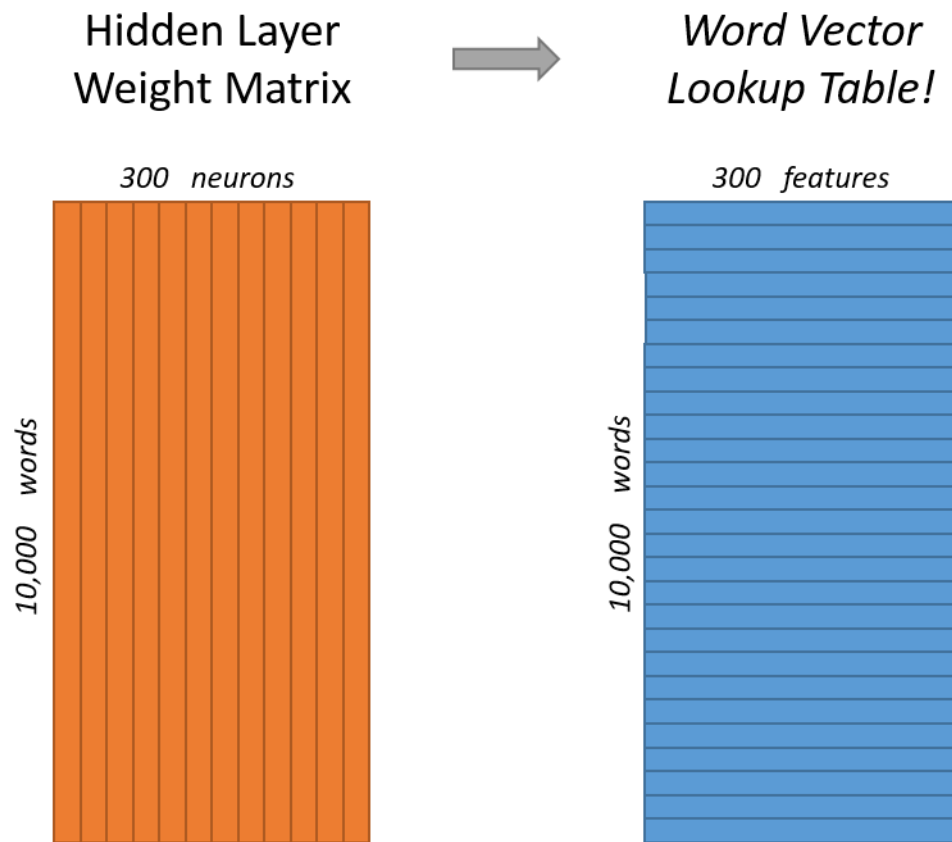
... "able"



... "zone"

10,000  
neurons

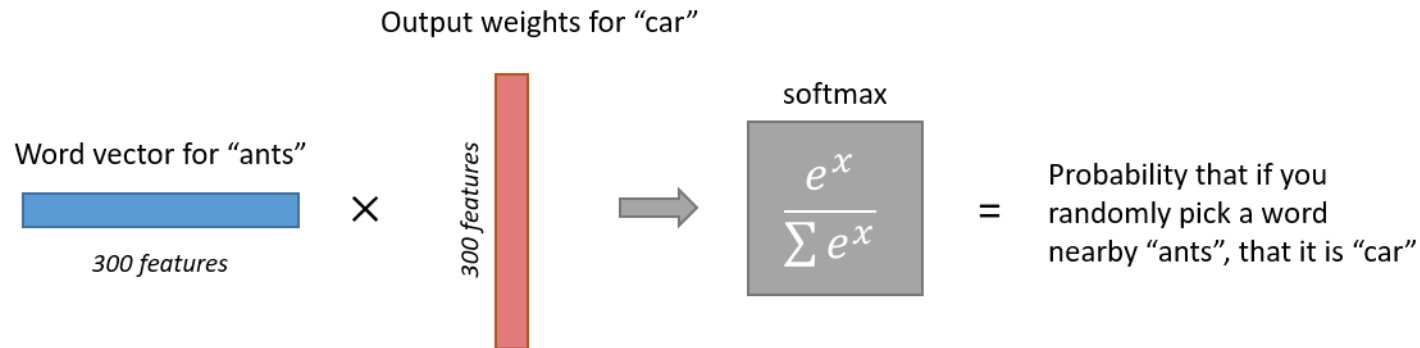
If you look at the *rows* of this weight matrix, these are actually what will be our word vectors!



**Teaching note:** This is exactly Exercise 2b: a  $|V| \times 300$  matrix, one row per vocabulary word — looking up a one-hot vector just selects that word's row.

# The Output Layer

Here's an illustration of calculating the output of the output neuron for the word "car".



The output layer is a *softmax* regression classifier: each output neuron (one per word in our vocabulary!) will produce an output between 0 and 1, and the sum of all these output values will add up to 1.

Each output neuron has a weight vector which it multiplies against the word vector from the hidden layer, then it applies the function  $\exp(x)$  to the result. Finally, in order to get the outputs to sum up to 1, this result is divided by the sum of the results from *all* 10,000 output nodes. The results is a probability value.

# Intuition

- If two different words have very similar “contexts” (that is, what words are likely to appear around them), then our model needs to output very similar results for these two words. And one way for the network to output similar context predictions for these two words is if the word vectors are similar. So, if two words have similar contexts, then our network is motivated to learn similar word vectors for these two words!
- And what does it mean for two words to have similar contexts? I think you could expect that synonyms like “intelligent” and “smart” would have very similar contexts. Or that words that are related, like “engine” and “transmission”, would probably have similar contexts as well.

# Next Steps to Word2Vec

Huge neural network!

Word vectors with 300 components, and a vocabulary of 10,000 words. A neural network with two weight matrices—a hidden layer and output layer. Both of these layers would have a weight matrix with  $300 \times 10,000 = 3$  million (3M) weights each.

Running gradient descent on a neural network that large is going to be slow. We need a huge amount of training data in order to tune that many weights and avoid over-fitting. millions of weights times billions of training samples means that training this model is going to be very hard.

Word2Vec addresses these issues as follows in Mikolov et al., 2013:

1. Treating common word pairs or phrases as single “words” in their model (remember: Mutual Information).
2. Subsampling frequent words to decrease the number of training examples (balance the training set).
3. Modifying the optimization objective with a technique they called “Negative Sampling”, which causes each training sample to update only a small percentage of the model’s weights.



# Why was Word2Vec so popular

Revolutionized the use of word embedding's by using a continuous bag of words and skip-grams to derive high quality word embedding's.

**Why:** unexpected side effect was compositionality: algebraic operations on word vectors result in a vector that is a semantic composite:

man + royal = king  
men – king = women –  
country – capital ...

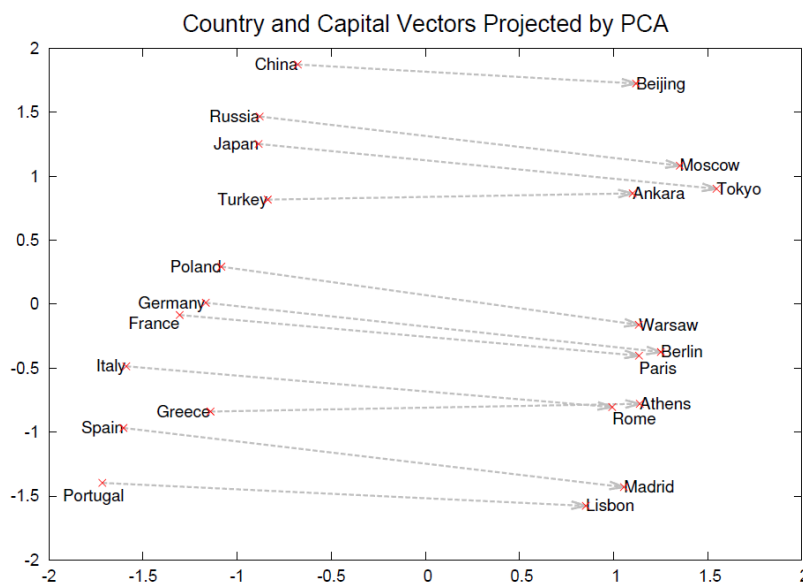


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

# Bias in Word2Vec

$$\vec{\text{man}} - \vec{\text{woman}} \approx \vec{\text{king}} - \vec{\text{queen}}$$

$$\vec{\text{man}} - \vec{\text{woman}} \approx \vec{\text{computer programmer}} - \vec{\text{homemaker}}.$$

## Extreme *she* occupations

- |                 |                       |                        |
|-----------------|-----------------------|------------------------|
| 1. homemaker    | 2. nurse              | 3. receptionist        |
| 4. librarian    | 5. socialite          | 6. hairdresser         |
| 7. nanny        | 8. bookkeeper         | 9. stylist             |
| 10. housekeeper | 11. interior designer | 12. guidance counselor |

## Extreme *he* occupations

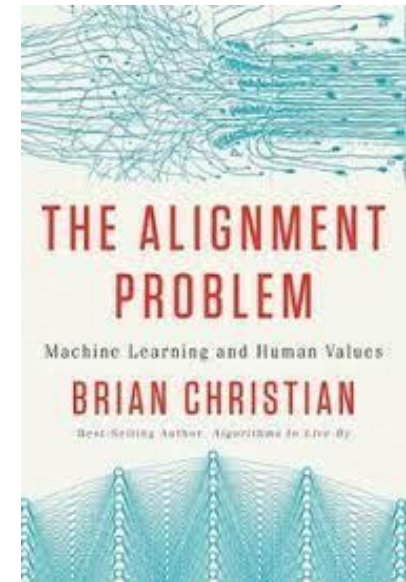
- |                |                   |                |
|----------------|-------------------|----------------|
| 1. maestro     | 2. skipper        | 3. protege     |
| 4. philosopher | 5. captain        | 6. architect   |
| 7. financier   | 8. warrior        | 9. broadcaster |
| 10. magician   | 11. fighter pilot | 12. boss       |

They typed:

doctor - man + woman

The answer came back:

nurse



# Language Sensitivity

- Word2Vec and GloVe (a similar model to Word2Vec) are completely language dependent.
- Word embeddings for English WILL NOT work for other languages.
- Check <https://research.fb.com/fasttext/> for word embeddings in other languages.
- Word2Vec and GloVe require that languages are completely separated.
- Word2Vec and GloVe may work well for multi-lingual collections if you calculate custom word embeddings for that collection, but in order to do so, you need lot's of data!
- Google's Word2Vec model for English, which includes word vectors for a vocabulary of 3 million words and phrases has been trained on roughly 100 billion words from the Google News dataset!

## Additional Limitations of Word2Vec (& GloVe a popular variant ...)

- Cannot deal with *out-of-vocabulary words*. Solution: Preprocess and assign left-over codes to these. Miss the semantic context.
- Combined high-frequent combinations used in Word2Vec (remember the optimization *Treating common word pairs or phrases as single “words”* as we discussed last course) does not result in compositionality for the individual terms. Solution: train word pairs also as individual tokens, but then harder to deal with frequent multi-token words such as *New York*.
- The small windows of surrounding words sometimes leads to problems for high frequent words: *bad* and *good* often occur in the same context leading to similar encodings resulting in problems in sentiment and emotion mining due to polarity confusion. Solution: preprocessing of negations and polarity.
- Word embedding's assign a unique vector to each textually unique word, so a word cannot have multiple meaning (*polysemy - homonyms*). Remember a word like *bank*. Solution: preprocess and replace homonyms with different tokens for each meaning: *bank1, bank2, bank3, bank4, ...*

# Dimensionality Popular Embeddings

| Model Category    | Model Name             | Dimensionality (dmodel) | Notes                                                                           |
|-------------------|------------------------|-------------------------|---------------------------------------------------------------------------------|
| Static Embeddings | Word2Vec (W2V)         | 300                     | The classic Google News vectors. <b>512</b> is common in custom shallow models. |
| Encoder (BERT)    | BERT-Base              | 768                     | 12 heads $\times$ 64 dim per head.                                              |
|                   | BERT-Large             | 1024                    | 16 heads $\times$ 64 dim per head.                                              |
| Modern Encoder    | ModernBERT-Base        | 768                     | Optimized for long context (8k+) and speed.                                     |
|                   | ModernBERT-Large       | 1024                    | Direct, more efficient drop-in for BERT-Large.                                  |
| OpenAI Models     | ada-002                | 1536                    | The long-standing standard for OpenAI embeddings.                               |
|                   | text-embedding-3-small | 1536                    | High performance, lower cost.                                                   |
|                   | text-embedding-3-large | 3072                    | Highest performance; supports Matryoshka truncation.                            |
| Generative (LLMs) | GPT-2                  | 768                     | Smallest variant.                                                               |
|                   | Llama 3 (8B)           | 4096                    | 32 heads $\times$ 128 dim per head.                                             |
|                   | GPT-3 / GPT-4          | 12288                   | Estimated internal hidden size for largest dense layers.                        |



## Bi-Directional Context to deal better with homonyms

“I accessed the bank account”

unidirectional contextual model would represent “bank” based on “I accessed the” but not “account.”

bi-directional contextual model represents “bank” using both its previous and next context — “I accessed the ... account”

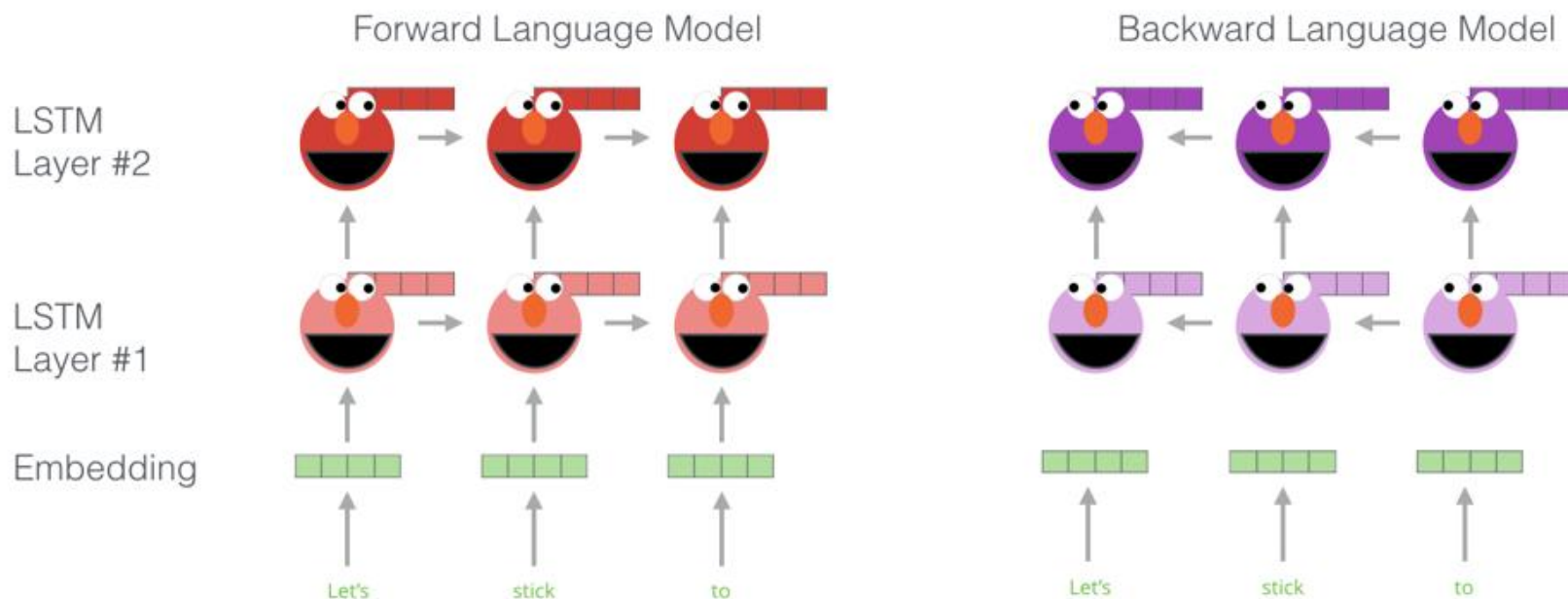
Both ELMo and BERT are bi-directional. ELMo is *shallow* bi-directional, BERT *deep* bi-directional.

# ELMo Word Embeddings (Allen NLP)

Instead of using a fixed embedding for each word, ELMo looks at the entire sentence before assigning each word in it an embedding. It uses a bi-directional LSTM trained on a specific task to be able to create those embeddings.

ELMo actually goes a step further and trains a bi-directional LSTM – so that its language model doesn't only have a sense of the next word, but also the previous word.

Embedding of “stick” in “Let’s stick to” - Step #1



# BERT Embeddings take a similar approach\*

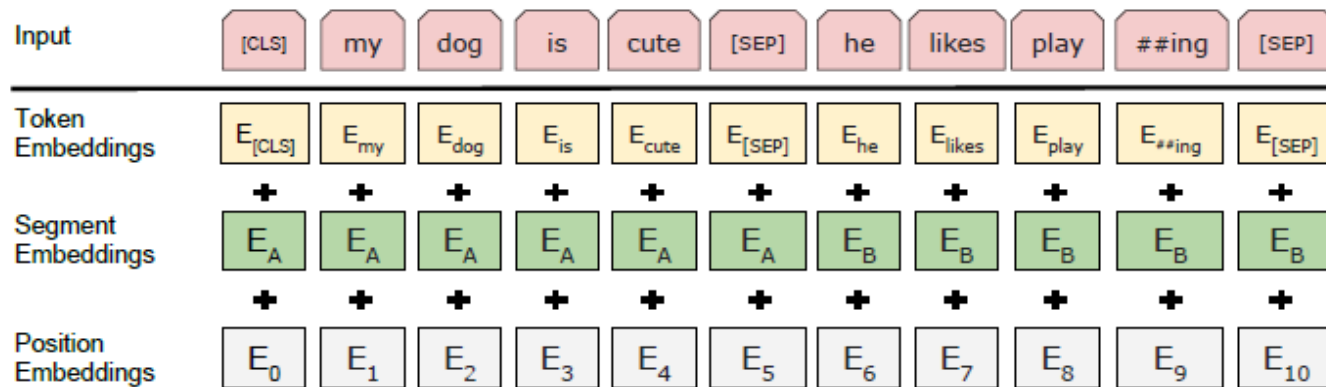


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

**Teaching note:** The three summed components: token embedding (which sub-word), segment embedding (which sentence, A or B), and position embedding (where in the sequence) — see Exercise 3b.

\* We will take a closer look at BERT in the lecture Deep-Learning for NLP – Part 2 and in the Deep Learning Tutorials.

# Context Sensitive Embeddings

## Sentences

"bank"  
"he eventually sold the shares back to the bank at a premium."  
"the bank strongly resisted cutting interest rates."  
"the bank will supply and buy back foreign currency."  
"the bank is pressing us for repayment of the loan."  
"the bank left its lending rates unchanged."  
"the river flowed over the bank."  
"tall, luxuriant plants grew along the river bank."  
"his soldiers were arrayed along the river bank."  
"wild flowers adorned the river bank."  
"two fox cubs romped playfully on the river bank."  
"the jewels were kept in a bank vault."  
"you can stow your jewellery away in the bank."  
"most of the money was in storage in bank vaults."  
"the diamonds are shut away in a bank vault somewhere."  
"thieves broke into the bank vault."  
"can I bank on your support?"  
"you can bank on him to hand you a reasonable bill for your services."  
"don't bank on your friends to help you out of trouble."  
"you can bank on me when you need money."  
"i bank on your help."

The different meanings of the word "bank" will get very different vectors, depending on this context. So, the "financial bank" will be closer to other financial words than the "river bank". This is essential for good language understanding!

**Teaching note:** This is the direct fix for both WordNet's hash-lookup problem (slide 25) and Word2Vec's one-vector-per-word limit (slide 27).

mouse (rodent)

mouse (computer)

So, we need different vectors (directions) for different meanings.

# BERT (Sub-)words

Internally, BERT stores 3 types of (sub)words:

- 1) “Context-free” pre-trained embeddings
- 2) “Context-based” pre-trained embeddings
- 3) “Context-averaged” pre-training embeddings

Sub-words:

The original BERT vocabulary is fixed at 30.000 tokens. Words that are not part of the core vocabulary are represented as subwords and characters: notice how the word “embeddings” can be represented: ['em', '##bed', '##ding', '##s']

# Word Piece Tokenizer

- The Word Piece tokenizer is a subword tokenization algorithm widely used in modern Natural Language Processing (NLP) models, particularly in transformer-based architectures such as BERT and GPT.
- It effectively balances vocabulary size and the ability to represent rare and unseen words while improving computational efficiency.
- Word Piece is language and domain dependent.

# The WordPiece Tokenizer

- BERT uses what is called a WordPiece tokenizer.
  - Words are tokenized either into the full forms if it is a known word (part of a 30k vocabulary, then one word becomes one token), or
  - Words are tokenized into word pieces (if it is not part of the 30k vocabulary), then one word can be broken into multiple tokens.

*We will discuss this in more detail in the tutorial*

**Teaching note:** Why it matters: any word can be built from known sub-word pieces, so the model never hits a true “unknown word” wall. However: is the final vector representation correct?

| Word         | Token(s)                     |
|--------------|------------------------------|
| surf         | ['surf']                     |
| surfing      | ['surf', '##ing']            |
| surfboarding | ['surf', '##board', '##ing'] |
| surfboard    | ['surf', '##board']          |
| snowboard    | ['snow', '##board']          |
| snowboarding | ['snow', '##board', '##ing'] |
| snow         | ['snow']                     |
| snowing      | ['snow', '##ing']            |

# Advantages of Word Piece

Word Piece tokenization has several advantages:

- 1. Efficient Vocabulary Size:** Reduces the vocabulary size compared to word-level tokenization while preserving word representation quality.
- 2. Handles Out of Vocabulary (OOV) Words:** Unknown words are decomposed into known subwords, improving model robustness.
- 3. Better Morphological Representation:** Works well with morphologically rich languages by capturing subword structures.
- 4. Improves Downstream NLP Performance:** It enhances transformer-based models by ensuring that embeddings are well-distributed across different words.

# Limitations of WordPiece

While WordPiece is highly effective, it does have limitations:

1. **Loss of Interpretability:** Tokenizing into subwords can make text less human-readable.
2. **Fixed Vocabulary:** Even though it handles OOV words, new words appearing in domain-specific tasks might still require retraining the tokenizer.
3. **Training Complexity:** Constructing an optimal vocabulary requires computationally expensive corpus analysis.
4. **Limited Context for Sub-Words:** too creative use of subwords may lead to non-related tokens getting similar embeddings or missing relevant contexts.
5. There is always the risks of **unknown characters, complex tokens** such as Chemical Formulas or DNA/RNA that requires special preprocessing, or specific language dependencies (direction, hyphens, 's, etc.) If such exceptions are high frequent, then such tokens will get irrelevant embeddings, similar to the random codes in a traditional tokenizer.

# When Using WordPiece ...

1. Measure # of OOV tokens
2. Measure # of tokens approximated with Sub-words
3. Check for # non recognized characters
4. Check for # single or double characters and digits (or combinations)

If these # are disproportional, you may have to finetune WordPiece for your specific data set. There are already many specialized tokenizers for Chemicals, DNA, RNA, etc.

But in eCommerce applications, you may have to do something for article numbers or specific serial numbers.



# The Original Embeddings were still limited

- Highly language sensitive (EN did not work for NL or GE)
- No support for homonyms
- Static vocabulary, many Out of Vocabulary (OOV) tokens
- Highly frequent tokens in similar context often get similar embeddings (good-bad, high-low, ...)

# Multilingual BERT

MultilingualBERT (M-BERT), pre-trained from monolingual corpora in 104 languages.

Surprisingly good at zero-shot cross-lingual model transfer, in which task-specific annotations in one language are used to fine-tune the model for evaluation in another language.

Even to languages in different scripts and that the model can find translation pairs.

M-BERT creates multilingual representations, but that these representations exhibit systematic deficiencies affecting certain language pairs.

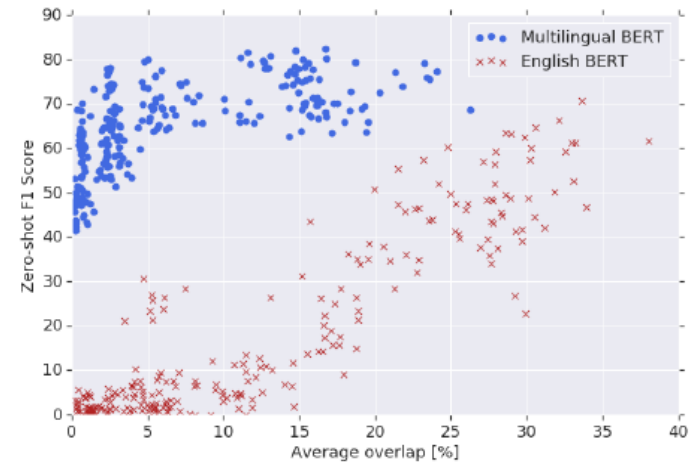


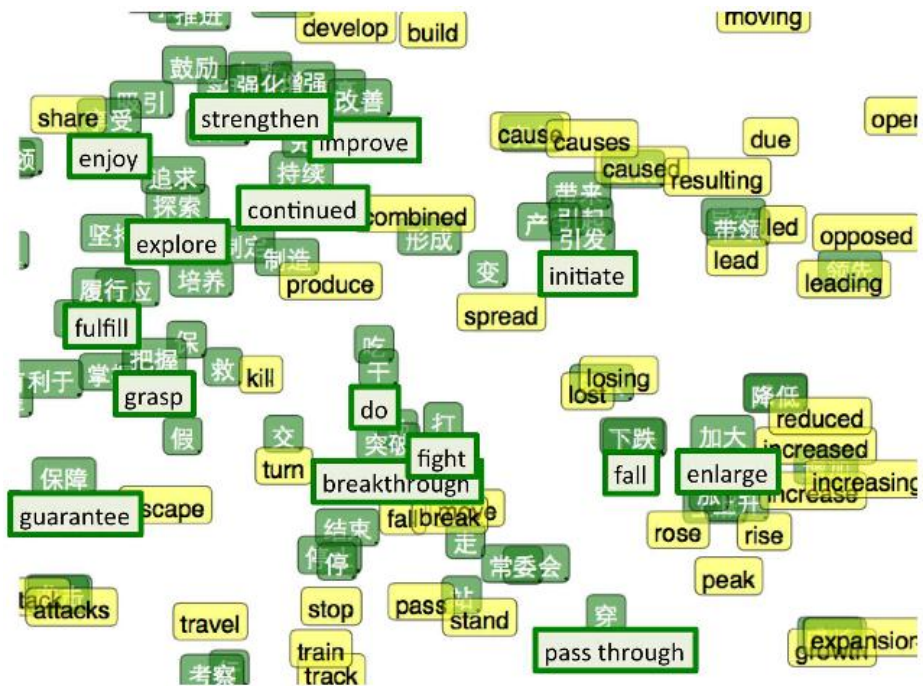
Figure 1: Zero-shot NER F1 score versus entity word piece overlap among 16 languages. While performance using EN-BERT depends directly on word piece overlap, M-BERT's performance is largely independent of overlap, indicating that it learns multilingual representations deeper than simple vocabulary memorization.

Source: How multilingual is Multilingual BERT? Telmo Pires, Eva Schlinger, Dan Garrette.

<https://arxiv.org/abs/1906.01502>

# Cross-Lingual, Multi-lingual ???

- Cross-lingual embeddings attempt to ensure that words that mean the same thing in different languages map to almost the same vector.
- Multilingual embeddings are happy if the embeddings work well in language A and work well in language B separately without any guarantees about interaction between different languages.



*We will discuss this in more detail in the tutorial*

# NLP/LLM = (mostly) Linear Algebra

$Ax = \lambda x$   
 $\uparrow$   $n \times n$  Matrix       $\uparrow$  scalar       $\uparrow$  "Eigen Value"  
 "Eigen Vector"

same thing  $\rightarrow (A - \lambda I)x = 0$   
 $\det(A - \lambda I) = 0 \leftarrow$  "Characteristic equation of A"  
 $(A - \lambda I)$  is singular  
 $(A - \lambda I)$  is not invertable

$P(\lambda) = \det(A - \lambda I)$   
 "Characteristic Polynomial"

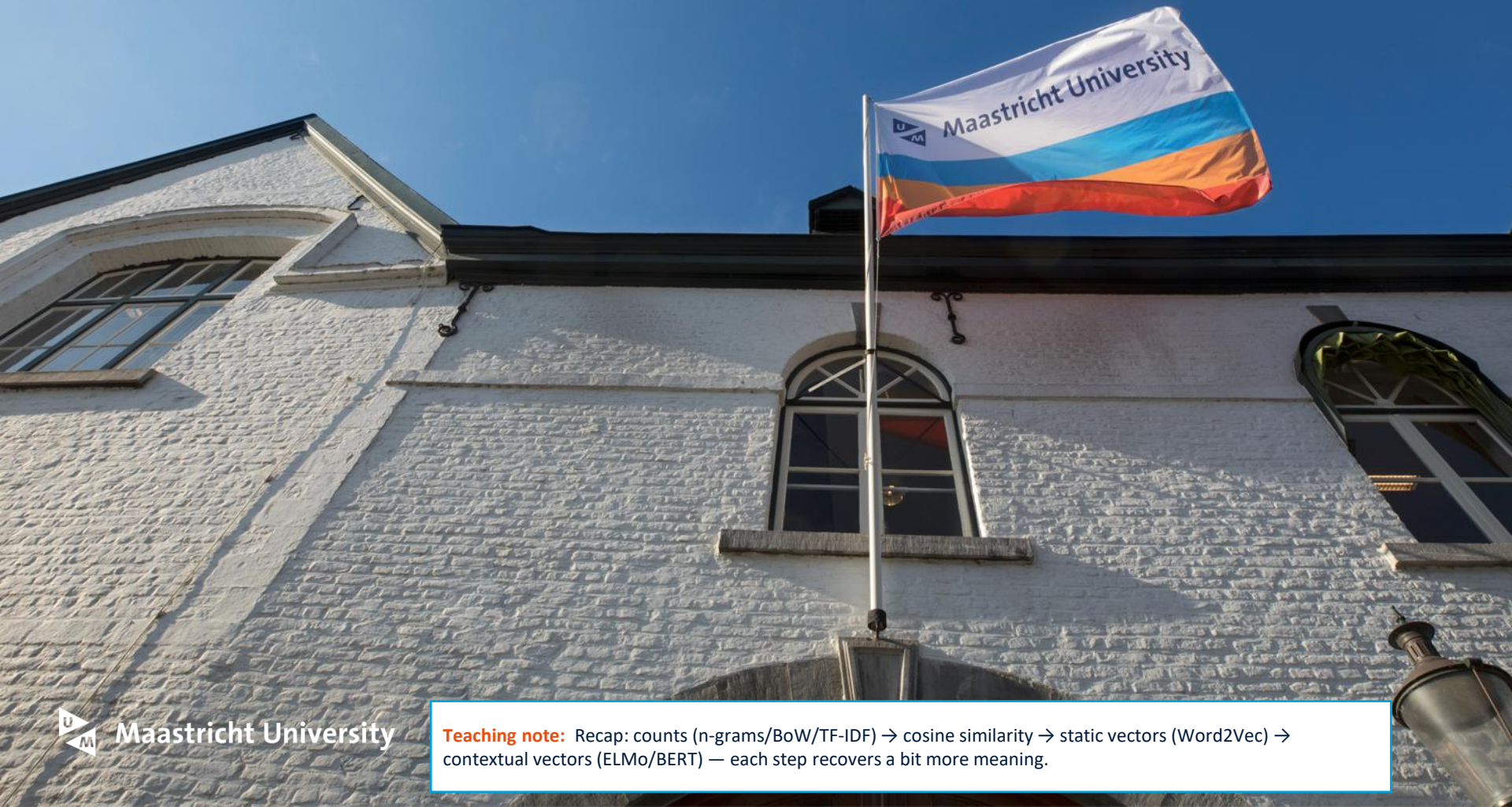
$A = \begin{bmatrix} 4 & 8 \\ 6 & 26 \end{bmatrix}$      $A - I = \begin{bmatrix} 4 & 8 \\ 6 & 26 \end{bmatrix} - \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 8 \\ 6 & 25 \end{bmatrix}$   
 $\lambda I = \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}$   
 $A - \lambda I = \begin{bmatrix} 4 & 8 \\ 6 & 26 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} = \begin{bmatrix} 4 - \lambda & 8 \\ 6 & 26 - \lambda \end{bmatrix}$

$\det(A - \lambda I) = \begin{vmatrix} 4 - \lambda & 8 \\ 6 & 26 - \lambda \end{vmatrix} = (4 - \lambda)(26 - \lambda) - (8)(6) = 104 - 30\lambda + \lambda^2 - 48 = \lambda^2 - 30\lambda + 56 = (\lambda - 28)(\lambda - 2) = 0$   
 $\lambda_1 = 28$   
 $\lambda_2 = 2$

$N(A - \lambda_2 I) = \left[ \begin{array}{cc|c} 4 - \lambda_2 & 8 & 0 \\ 6 & 26 - \lambda_2 & 0 \end{array} \right] = \left[ \begin{array}{cc|c} 4 - 2 & 8 & 0 \\ 6 & 26 - 2 & 0 \end{array} \right] = \left[ \begin{array}{cc|c} 2 & 8 & 0 \\ 6 & 24 & 0 \end{array} \right] \xrightarrow{R1/2} \left[ \begin{array}{cc|c} 1 & 4 & 0 \\ 6 & 24 & 0 \end{array} \right] \xrightarrow{R2 - 6R1} \left[ \begin{array}{cc|c} 1 & 4 & 0 \\ 0 & 0 & 0 \end{array} \right]$

$x_1 + 4x_2 = 0 \rightarrow x_1 = -4x_2 \rightarrow \begin{bmatrix} -4x_2 \\ x_2 \end{bmatrix} \rightarrow \begin{bmatrix} -4x \\ x \end{bmatrix} \rightarrow x \begin{bmatrix} -4 \\ 1 \end{bmatrix}$   
 Eigenspace  $\rightarrow$  Eigen Vector

# Questions?



Maastricht University

**Teaching note:** Recap: counts (n-grams/BoW/TF-IDF) → cosine similarity → static vectors (Word2Vec) → contextual vectors (ELMo/BERT) — each step recovers a bit more meaning.

# Exercise 1

Consider a collection of  $N = 10,000$  documents. You are analyzing two terms:

- Term A:  $df_A = 10$  (appears in 10 documents)
- Term B:  $df_B = 1,000$  (appears in 1,000 documents)  
(Use  $\text{idf}_t = \log_{10}(N/df_t)$  and log-frequency term weighting  $w_{t,d} = 1 + \log_{10}(\text{tf}_{t,d})$  for  $\text{tf}_{t,d} > 0$ , matching the lecture slides.)
- Calculate the idfweights for both Term A and Term B. If Document  $d_1$  contains Term A with a raw frequency count of  $\text{tf}_{A,d_1} = 100$ , what is the final tf-idfweight of Term A in document  $d_1$ ?
- Suppose document  $d_2$  is created by duplicating all the contents of  $d_1$  (so every word occurs exactly twice as often).
  - Explain why the Euclidean distance between vectors  $\vec{v}(d_1)$  and  $\vec{v}(d_2)$  is greater than 0.
  - What is the cosine similarity  $\cos(\theta)$  between  $\vec{v}(d_1)$  and  $\vec{v}(d_2)$ , and why does this make angle/cosine comparison superior to Euclidean distance for document retrieval?
- Why does the Bag of Words (BoW) assumption fail to distinguish between "*Jane hit Jon*" and "*Jon hit Jane*", and why are raw document vectors in  $\mathbb{R}^{|V|}$  extremely sparse (often  $< 0.01\%$  non-zero)?

## Exercise 2

Word2Vec represents words as dense continuous vectors rather than sparse, high-dimensional one-hot vectors.

a. Given the training sentence:

- "The quick brown fox jumps over the lazy dog."
- List all positive training sample pairs (center\_word, context\_word) generated when the sliding window selects "**brown**" as the target center word with a context window radius of 2 words on either side.

b. In the Skip-gram architecture shown on slide 22, suppose the vocabulary size is  $|V| = 10,000$  and the hidden linear layer has  $N = 300$  neurons:

- What are the dimensions of the hidden layer weight matrix?
- Explain how this weight matrix directly acts as a lookup table for the dense 300-dimensional word embeddings.

c. One key discovery of Word2Vec was semantic compositionality via vector arithmetic (e.g.,  $\vec{v}_{\text{king}} - \vec{v}_{\text{man}} + \vec{v}_{\text{woman}} \approx \vec{v}_{\text{queen}}$ ). What fundamental limitation prevented static Word2Vec models from disambiguating homonyms like "*river bank*" versus "*money bank*"?

# Exercise 3

Compare the architectural differences between static word embeddings (Word2Vec) and contextualized transformer embeddings (BERT).

- Look at the embedding dimensions table on slide 19. What is the dimensionality of classic Word2Vec versus BERT-Base and modern large models (e.g., Llama 3 or text-embedding-3-large)? Why does modern NLP move from 300 dimensions up to several thousands?
- Slide 28 outlines that the input representation in BERT is the element-wise sum of three distinct embedding vectors. Name these three components and briefly state what information each component provides to the model.
- In a downstream task containing the sentences:
  - *"The river flowed over the bank."*
  - *"The bank left its lending rates unchanged."*
  - Explain how the resulting vector representation of the token "bank" differs between Word2Vec and BERT, and describe how self-attention enables BERT to separate the vectors geometrically in latent space.

# Discussion Lecture: Text Representation



Maastricht University

**Teaching note:** Use the ideas from the lecture (TF-IDF, cosine, Word2Vec, BERT) as ammunition — there's no single correct answer to these.

# On Geometry, Angles, and the Nature of "Meaning"

In both TF-IDF and dense embedding models, we evaluate whether two texts mean the same thing by computing the cosine of the angle between their vector representations.

If we append a document to itself, the angle remains zero ( $\cos = 1$ ), which intuitively captures identical topic proportions.

But consider the two sentences: "Jane hit Jon" and "Jon hit Jane". In standard Bag-of-Words and vector-space models, their vectors are identical, giving an angle of zero. If two sentences with completely opposite real-world events occupy the exact same point in vector space, are geometric angles actually measuring 'meaning' or merely lexical co-presence?

At what point does spatial geometry fail to represent logic?

## On Word2Vec Compositionality: True Reasoning or Statistical Mirage?

One of the most famous results of Word2Vec is linear compositionality—that vector arithmetic seemingly solves analogies like  $\vec{v}_{\text{king}} - \vec{v}_{\text{man}} + \vec{v}_{\text{woman}} \approx \vec{v}_{\text{queen}}$ .

Did the neural network discover an abstract conceptual ontology of gender and monarchy, or did it simply exploit statistical regularities in how words co-occur across a local sliding window?

If it is merely window statistics, why does linear additivity emerge so cleanly from a non-linear objective like Skip-gram?

# On Contextualization and the Polysemy Paradox

- In static embeddings like Word2Vec, the word "bank" has exactly one point in vector space, forcing financial and river contexts to collide into a single averaged compromise vector. BERT solves this by producing different vectors for "bank" depending on its sentence context.
- But if "bank" in a financial sentence gets a completely different 768-dimensional vector than "bank" in a geography sentence, does the underlying token "bank" still retain an intrinsic identity in the model?
- If an embedding is entirely defined by its context, how does a model know that two completely different vectors in different texts refer to the same base dictionary entity?